

Special Project

# Autonomous Indoor Delivery Robot

professor: 蔡英德

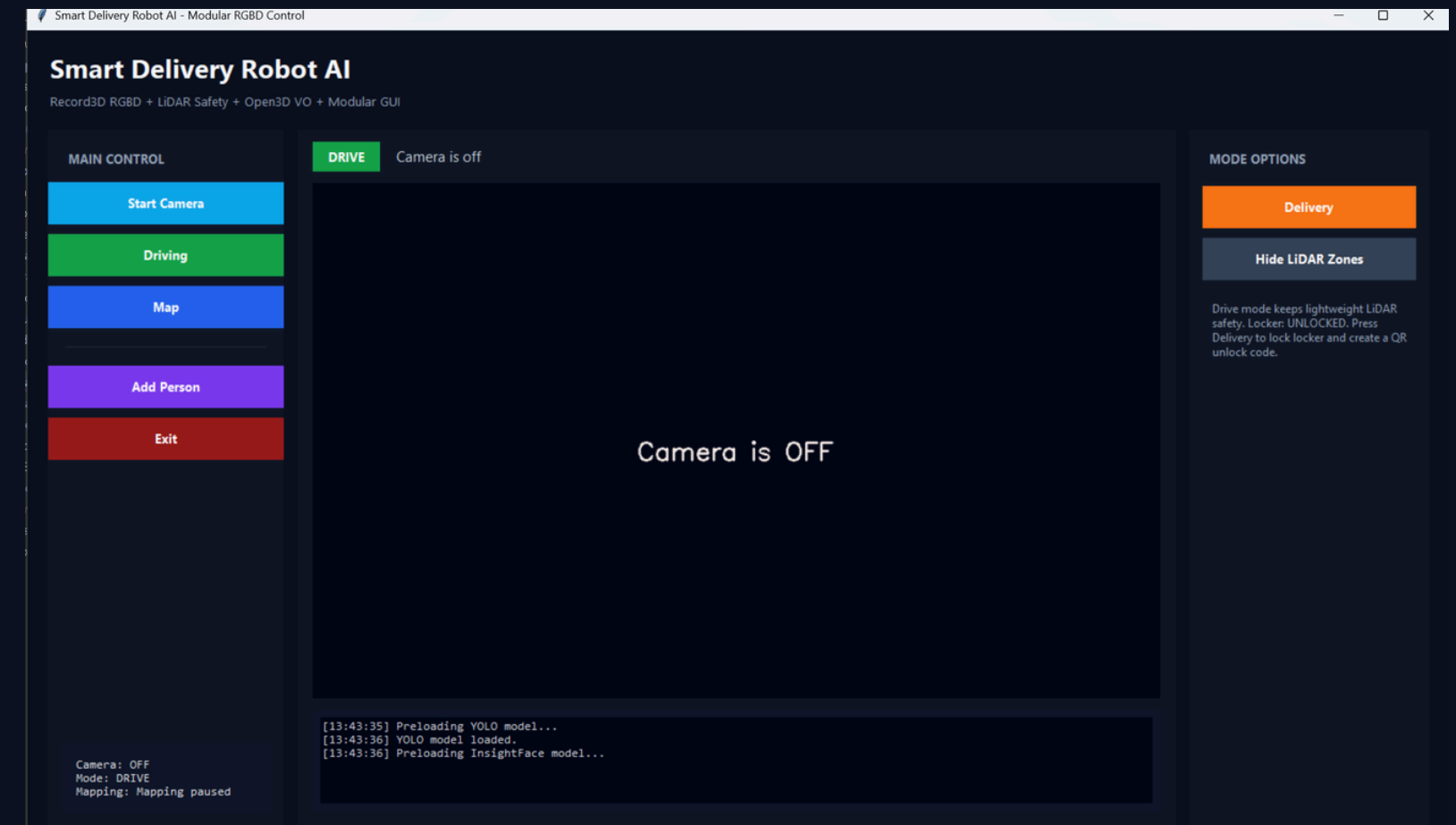
advisor: 蔡英德

topic: delivery self-driving car

members:

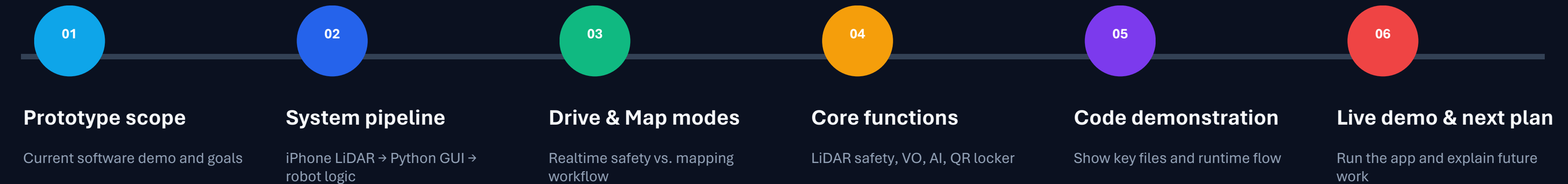
- 許俊成 - leader
- 張美勤
- 菲茲雷
- 特達

Current stage: software demo on PC with phone LiDAR  
Next stage: real robot with Jetson Xavier NX + ESP32



# Presentation Roadmap

The talk follows the software prototype flow, then ends with code demonstration and live demo.



### Demonstration code focus

I will connect visible GUI features to the files that control them.

`gui.py`  
main loop

`camera_record3d.py`  
RGBD stream

`lidar_safety.py`  
safety zones

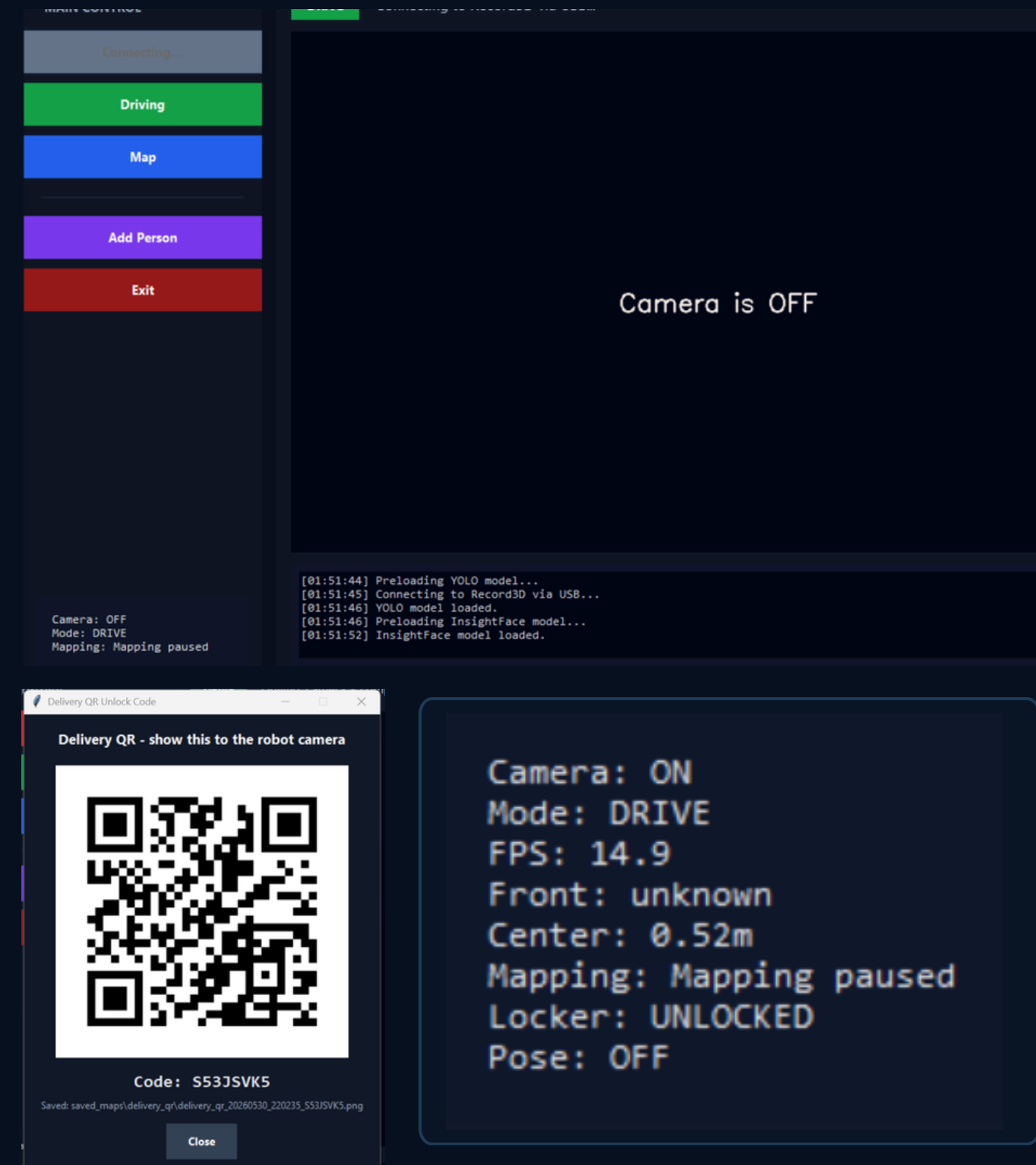
`mapping.py`  
occupancy grid

`locker.py`  
QR workflow

# Prototype Scope This Semester

What the current program can demonstrate clearly

- Connect iPhone LiDAR through Record3D USB
- Receive RGB frame + depth frame
- Show live distance values and LiDAR safety
- Switch between Drive mode and Map mode
- Create mapping preview from depth data
- Simulate QR-based locker unlocking
- Prepare modular software for real robot hardware



## Core message

This semester verifies the software pipeline before moving it to Jetson Xavier NX, ESP32, motors and a real locker.

# System Target

A software pipeline that can later run on real robot hardware

iPhone LiDAR  
+ Record3D

RGB frame  
Depth frame

Python GUI  
Laptop

Drive / Map  
Modes

Safety · Mapping  
QR · AI

Next semester  
Jetson + ESP32

## Current input

Phone RGBD stream gives the program both image and depth data.

## Current processing

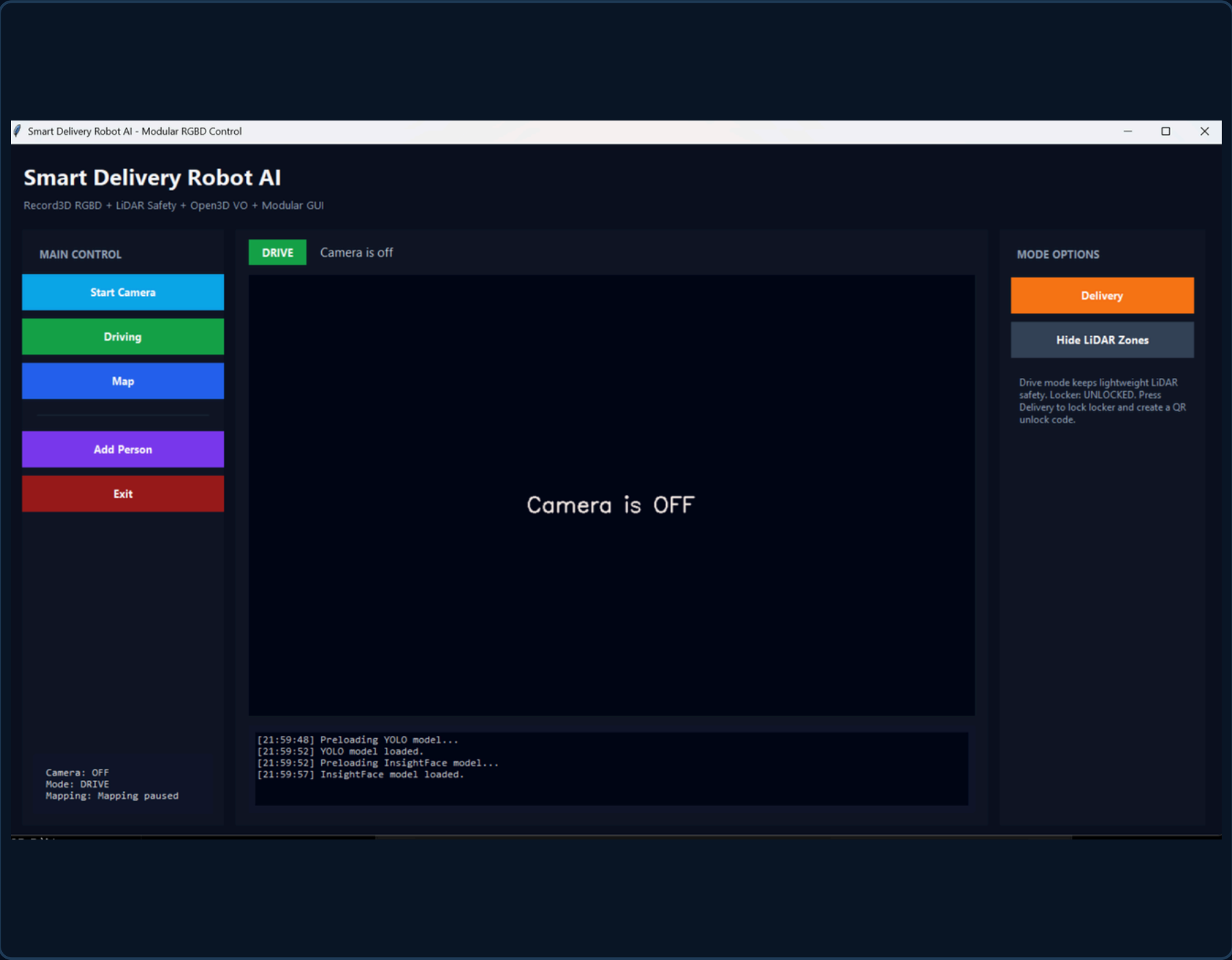
Python handles safety, mapping preview, QR scan and UI state.

## Future output

Jetson will run AI; ESP32 will control motors and locker hardware.

# Operator GUI

Designed for safe demo operation and debugging



## Left panel

Start/Stop Camera  
Drive / Map  
Add Person  
Exit

## Right panel

Mode-specific tools  
Delivery / hide Lidar zones  
VO / Mapping Save / Load Map

## Center preview

Camera + LiDAR overlay  
Map preview  
QR scan feedback

## Bottom log

Connection events  
Warnings  
Demo actions

## UI design rule

The operator sees only the controls needed for the current mode, so the demo is cleaner and less confusing.



# Live Values on the Screen

How to explain the numbers during the demo

```
Camera: ON  
Mode: DRIVE  
FPS: 14.9  
Front: unknown  
Center: 0.52m  
Mapping: Mapping paused  
Locker: UNLOCKED  
Pose: OFF
```

## Center

Distance from a small area in the middle of the camera image. It shows what is directly in front of the camera center.

## Front

Safety distance calculated from multiple front zones. It is more important for obstacle warning.

## Pose

Estimated position and orientation of the camera/robot. It can be OFF, VO or Record3D.

## FPS

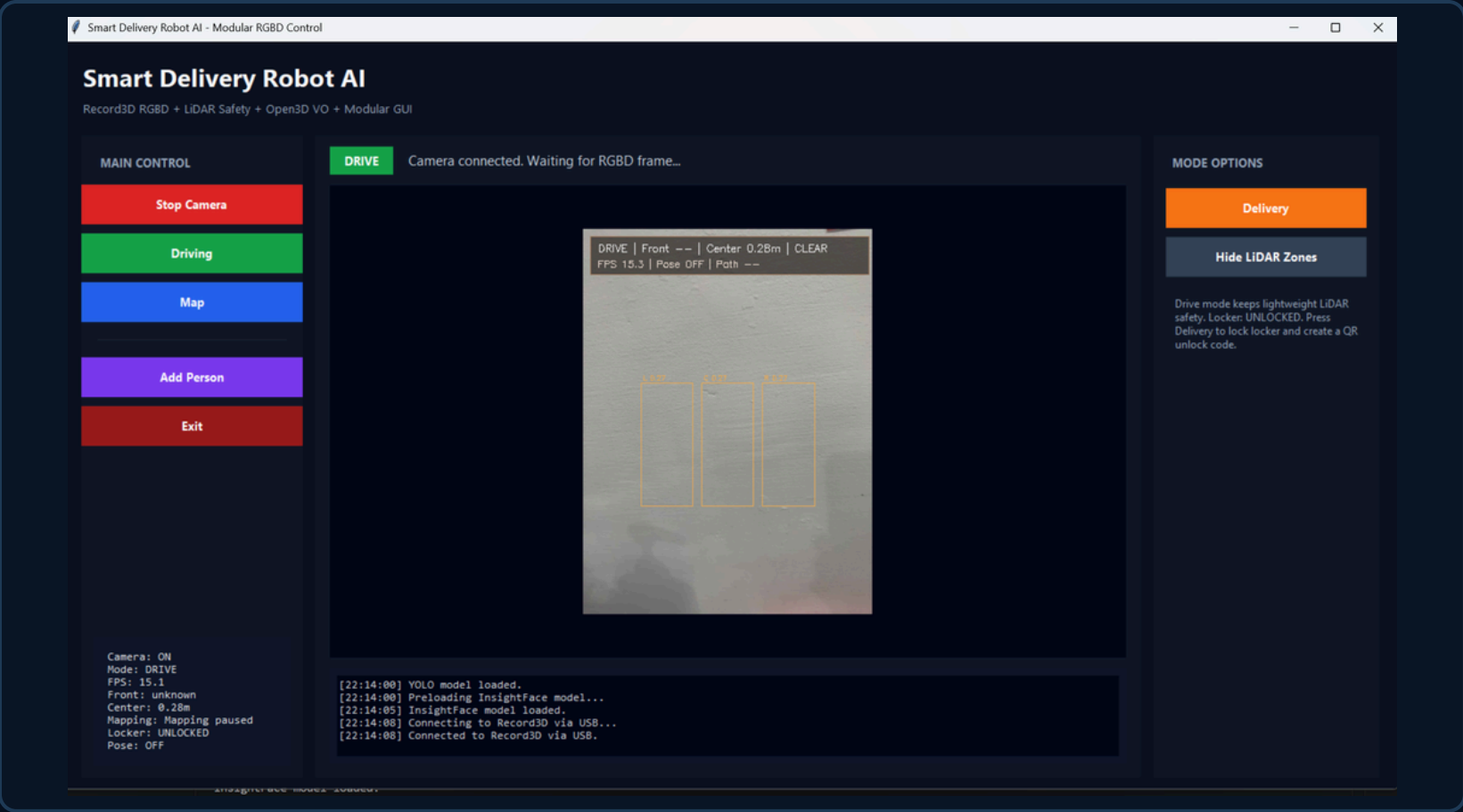
Frame processing rate. In this prototype, around 12–15 FPS can be normal because RGBD streaming and GUI rendering are heavy.

## What to say

Center is a simple depth reading. Front is used for safety. Pose is needed for global mapping, but it may be OFF when VO is not enabled or not successful.

# Drive Mode

Real-time camera, safety checks and delivery operation



## Main purpose

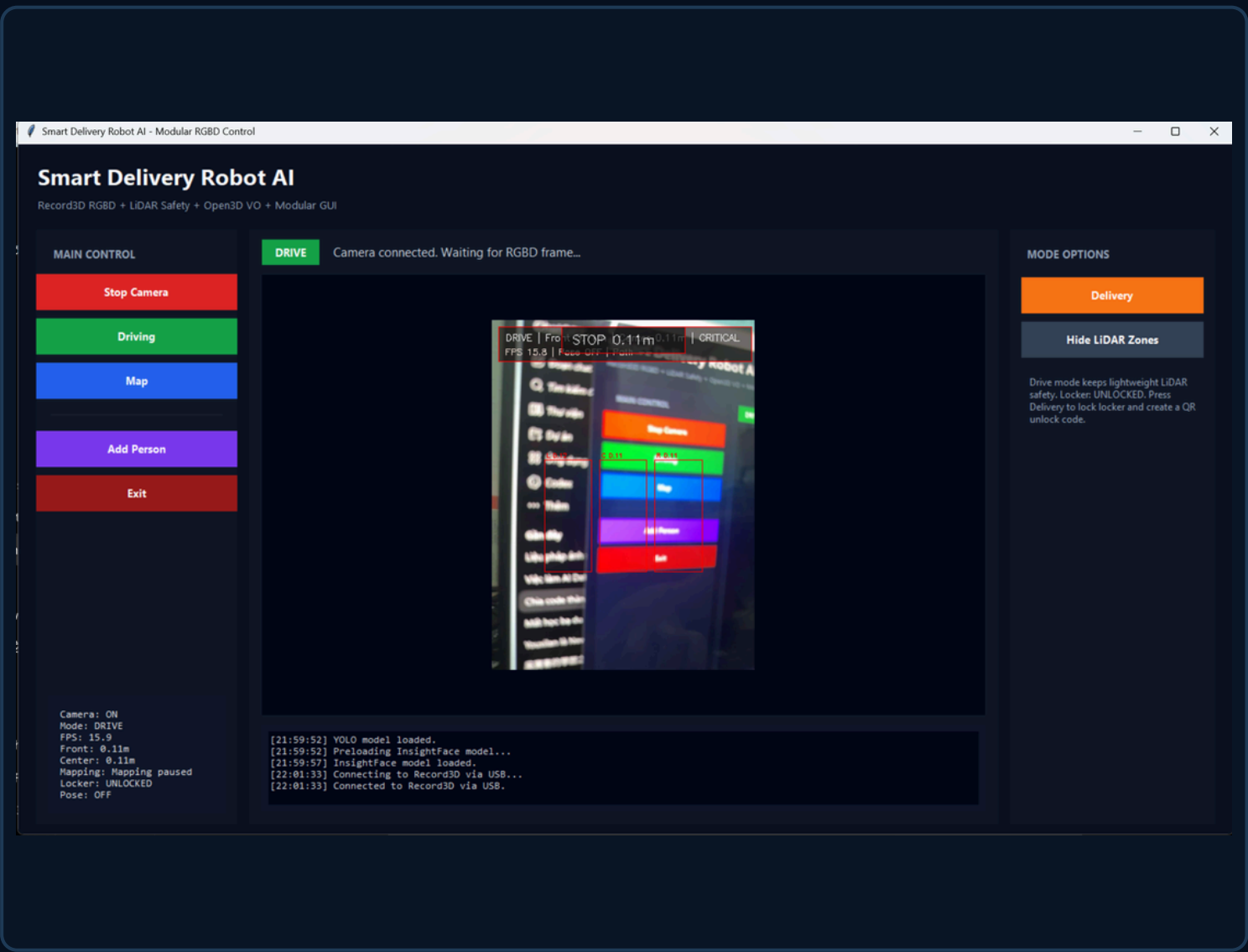
Low latency and safe front-distance checking.

## Heavy tasks

Mapping, VO and AI modules are reduced or disabled.

# LiDAR Safety

Depth frame becomes safety zones



Depth frame  
(distance matrix)

Left / Center / Right  
front zones

CLEAR /  
WARNING  
/ CRITICAL

## How it works

The program checks valid depth values inside each zone and keeps the closest stable distance.

## Why it matters

A delivery robot needs a quick safety layer before movement or path planning.

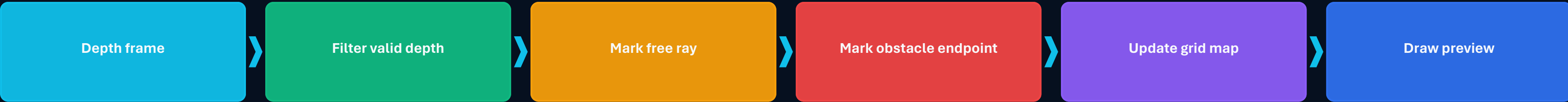
## Limitation

Phone LiDAR can be unstable at very close distance, reflective surfaces or weak-depth areas.



# Map Mode

Depth data is converted into an occupancy map



## Occupancy grid

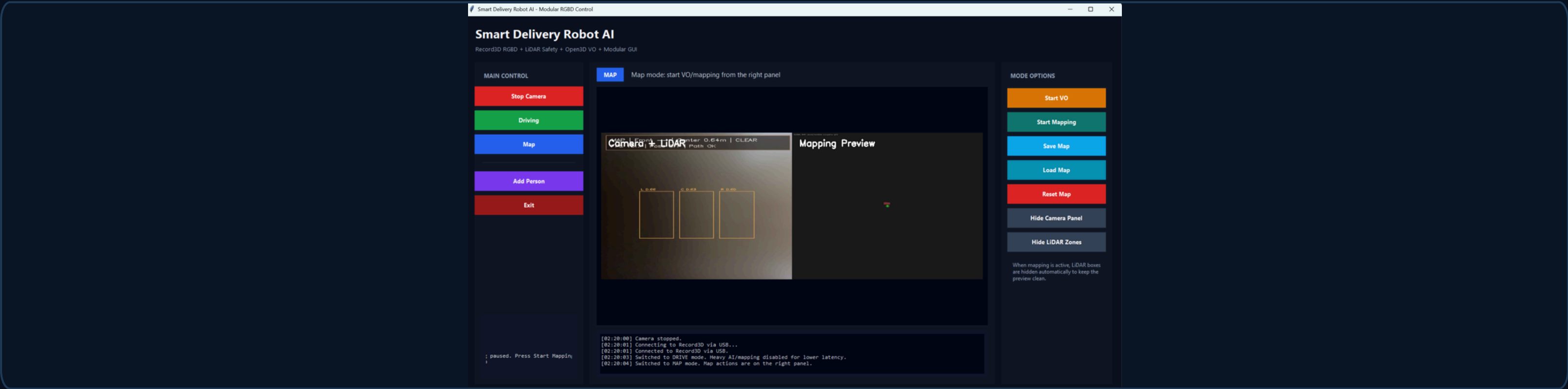
The map stores unknown, free and obstacle cells.

## Local map

Nearby environment around the robot; useful for short-term safety and path ideas.

## Global map

Larger map that remembers the environment already scanned.



# Visual Odometry

VO estimates camera movement between RGBD frames

Previous  
RGBD frame

Current  
RGBD frame

Open3D RGBD  
Odometry

Temporary  
Pose

## VO tells

How the camera has moved between frames.

## Mapping tells

What the environment looks like using depth.

## Current truth

VO is a demo function and can fail in flat, dark or low-texture scenes.

## Presentation wording

Depth tells where obstacles are. Pose tells where the camera/robot is located.

# VO is Not Full SLAM

A clear answer for professor questions

## Visual Odometry

Estimates motion between consecutive frames. It can provide temporary pose, but errors slowly accumulate.

## SLAM

Simultaneous Localization and Mapping: estimate where the robot is while building a map.

## Full SLAM adds

Loop closure, pose graph optimization, long-term map correction and global localization.

VO estimates motion



Mapping builds map



SLAM corrects drift

## Key sentence

Our current system is SLAM-like mapping, not complete SLAM. VO can support SLAM, but VO itself is not full SLAM.

# YOLO and InsightFace

Optional AI perception modules

## YOLO Detection

Detects objects or persons in the camera frame. It is useful for human/obstacle recognition, but it is computationally heavy.

## InsightFace Recognition

Detects faces, extracts face embeddings and compares them with the saved face database.

Camera frame

YOLO boxes

Face embedding

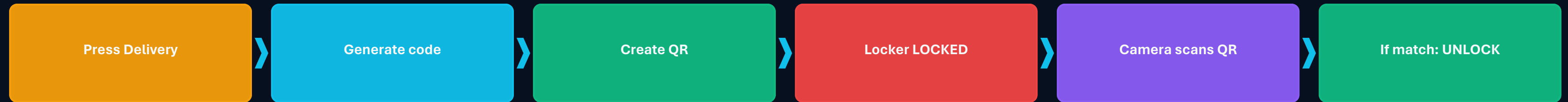
Name / Unknown

## Performance strategy

Models are preloaded at startup. In Drive mode they can be disabled or run less frequently, because low latency is more important than continuous AI detection.

# QR Locker Delivery Function

A software simulation of secure package pickup



## Current state

The locker is simulated in software: status changes between LOCKED and UNLOCKED.

## Future hardware

Python can send LOCK / UNLOCK commands to ESP32 through serial communication.

## Demo value

This connects the software demo to a real delivery scenario, not just camera and mapping tests.



# Modular Code Structure

Each file has a clear responsibility

## main.py

start program

## camera\_record3d.py

RGBD input

## visual\_odometry.py

pose estimate

## gui.py

main interface

## depth\_utils.py

depth values

## models.py

load AI

## config.py

settings

## lidar\_safety.py

front safety

## locker.py

QR locker

## app\_state.py

shared state

## mapping.py

grid map

## save\_load.py

save/load map

## Why this matters

The GUI coordinates modules, but camera, depth, safety, mapping, AI and locker logic are separated. This makes debugging and future hardware integration easier.

# Runtime Code Flow

What happens during each processing tick



```
if frame_id == last_camera_frame_id:
    return

if now - last_process_ts < 1 / MAX_PROCESS_FPS:
    return

frame, depth = resize_frame_and_depth(...)
center_depth = get_center_lidar_depth(depth)
safety_state, safety_distance = get_front_safety(depth)
```

## Anti-lag idea

Drop old frames instead of processing a backlog.

## Mode logic

Drive keeps safety. Map updates occupancy grid when enabled.

## Key explanation

The program prioritizes current frames. This is why frame dropping is better than trying to process every frame slowly.

# Current Limitations

What we should explain honestly

## Prototype limitations

- Laptop + iPhone LiDAR prototype only
- Robot chassis and motor control not completed yet
- Locker is simulated in software
- Record3D streaming may stay around 12–15 FPS

## Technical limitations

- Mapping is SLAM-like, not full SLAM
- VO can fail in flat or low-texture scenes
- YOLO/InsightFace are heavy models
- Tkinter video rendering is not ideal for high FPS

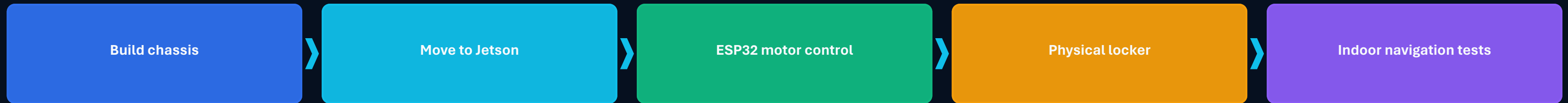
## How to say it

The current system does not claim to be a finished robot. It proves the core software pipeline and prepares the project for real hardware integration.



# Next Semester Development Plan

Move from software prototype to real robot implementation



## Hardware integration

3D printed chassis, motor driver, batteries, four motors and delivery box.

## Control integration

ESP32 receives motor and locker commands from the main computer.

## Navigation improvement

Improve mapping stability, path planning and real indoor delivery tests.

## Development direction

The current laptop demo is a validation stage. The next step is real movement, real locker control and repeated indoor testing.

# Conclusion and Q&A

What this semester successfully proves

## Completed this semester

RGBD camera connection  
Depth/LiDAR safety display  
Drive mode and Map mode  
Occupancy map preview  
QR locker workflow  
Modular code structure

## Prepared for next semester

Jetson Xavier NX migration  
ESP32 motor and locker control  
Physical chassis and locker  
Improved mapping / navigation  
Real indoor delivery test

# Thank you

Questions?

## Final message

This project is a working software prototype that proves the main perception, safety, mapping and delivery interaction flow.